

# Object Oriented Programming In Java

Saurabh Pandey  
Associate Professor  
Sitare University

# Java Exception Handling

**Objective:**

- Learn how Java handles runtime errors using *exceptions*, inheritance, and control flow statements.

# What is an Exception?

An **exception** is an event that disrupts the normal flow of a program.

Occurs **during runtime**, e.g.:

- Division by zero
- Accessing invalid array index
- File not found

Exception handling allows a program to **continue execution safely**.

# Why Exception Handling?

## Without Handling

---

Program crashes

---

Error messages confuse users

---

Harder debugging

## With Handling

---

Graceful recovery

---

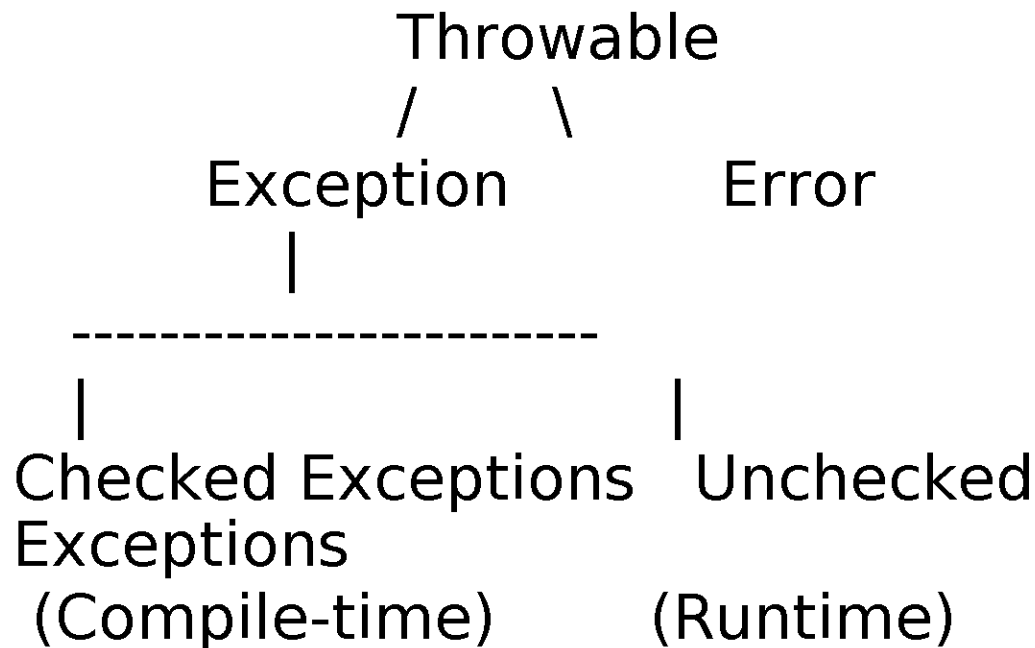
Clear error messages

---

Easier debugging

Improves program **stability**, **readability**, and **user experience**.

# Exception Hierarchy Diagram



# Checked vs Unchecked vs Error

| Category        | Checked Exceptions           | Unchecked Exceptions                         | Errors                                  |
|-----------------|------------------------------|----------------------------------------------|-----------------------------------------|
| When caught     | Compile-time                 | Runtime                                      | Runtime                                 |
| Package         | java.io, java.sql            | java.lang                                    | java.lang                               |
| Examples        | IOException,<br>SQLException | NullPointerException,<br>ArithmeticException | OutOfMemoryError,<br>StackOverflowError |
| Can be handled? | ✓ Yes                        | ✓ Yes                                        | ⚠ Usually No                            |



# Inheritance in Exception Handling

```
public class Throwable { }
```

```
public class Exception extends Throwable { }
```

```
public class IOException extends Exception { }
```

```
public class FileNotFoundException extends IOException { }
```

## **Inheritance Concept:**

- Child classes inherit behavior from parent exception classes.
- You can **catch parent exceptions** to handle all its child exceptions.

# try-catch-finally Flow

```
try {  
    int a = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero");  
} finally {  
    System.out.println("Always executed");  
}
```

Flow Diagram:

try → exception occurs → matching catch → finally → end

**finally** always executes (even if exception occurs).



# Multiple Catch & Exception Hierarchy

```
try {  
    String s = null;  
    System.out.println(s.length());  
}  
catch (ArithmeticException e) {  
    System.out.println("Math error");  
}  
catch (Exception e) {  
    System.out.println("General exception caught");  
}
```

## **Rule:**

Catch from **most specific** → **most general**

If you reverse, compiler error!

# throw & throws

- **throw**: explicitly throw an exception
- **throws**: declare exceptions that a method can throw

```
void validateAge(int age) throws Exception {  
    if (age < 18)  
        throw new Exception("Not eligible to vote");  
}
```

Helps in **propagating exceptions** up the call stack.

# Custom Exceptions

Create your own exception by extending `Exception` class.

```
class AgeException extends Exception {  
    AgeException(String msg) {  
        super(msg);  
    }  
}  
  
public class Test {  
    public static void main(String[] args) throws AgeException {  
        throw new AgeException("Age must be 18 or above!");  
    }  
}
```

# Exception Inheritance Diagram

